Laboratório Funcional · Aula 12

Heitor Saueressig Mello · RA 24042002-2 · Turma ESOFT6B
Gerado em 16/10/2026, 19:57:07 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora: depende do relógio do sistema. Cada vez que chama devolve um valor diferente, mesmo sem parâmetros.
registra: tem efeito colateral escreve na tela. Não devolve um valor útil, só imprime.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

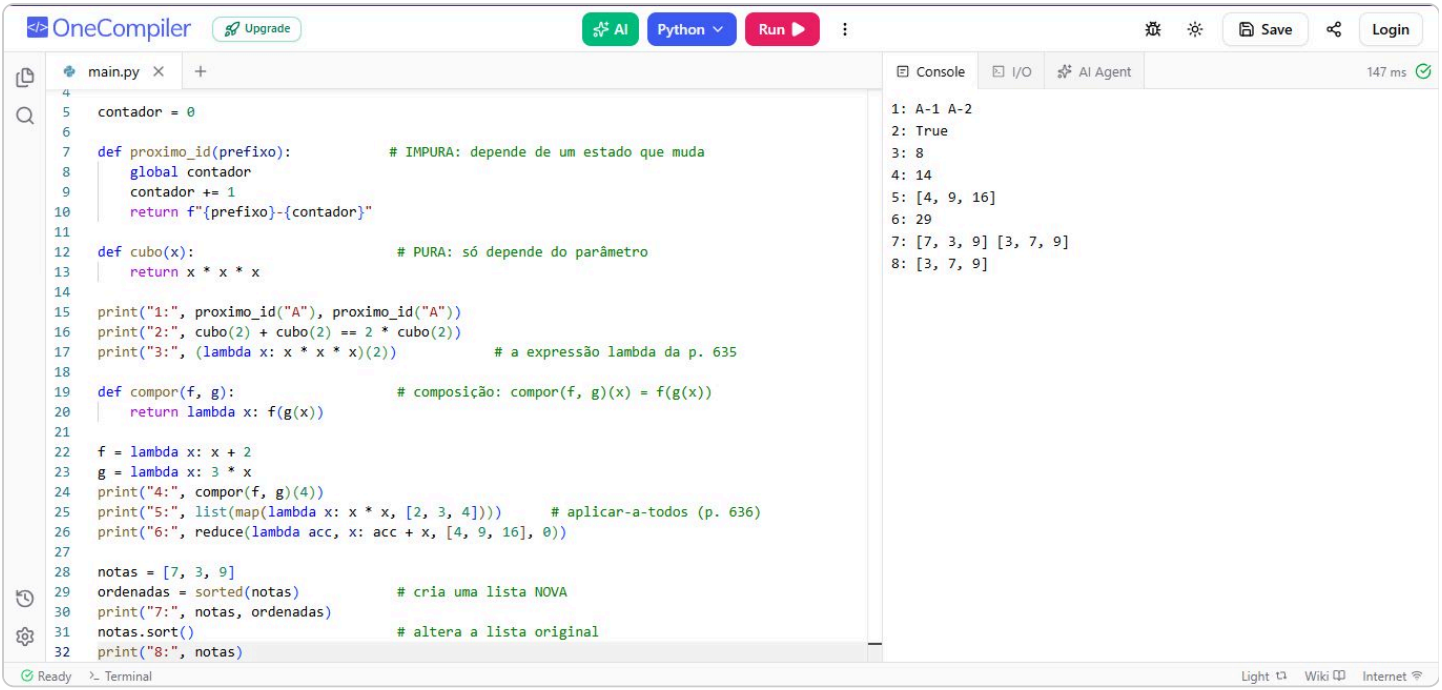
4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
Original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

Com compor(g, f):
4: 18
```




```python
contador = 0

def proximo_id(prefixo):            # IMPURA: depende de um estado que muda
    global contador
    contador += 1
    return f"{prefixo}-{contador}"

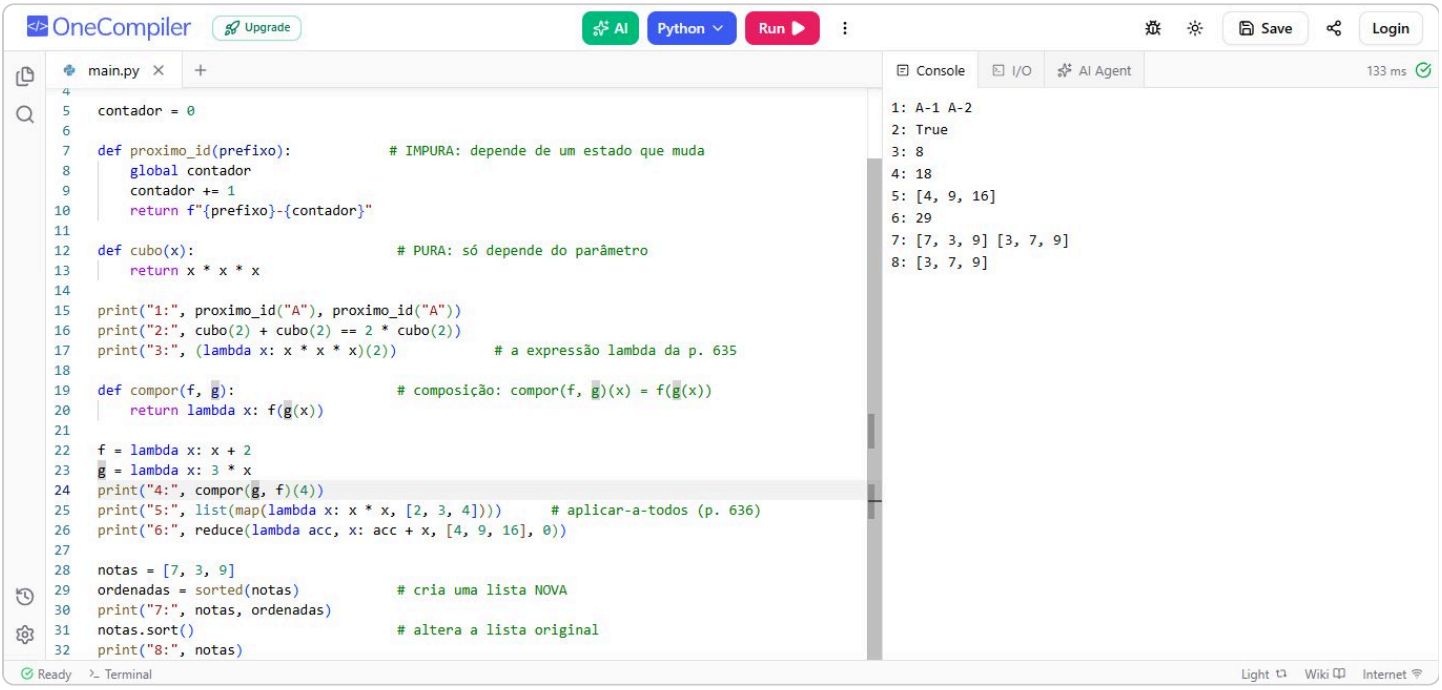
def cubo(x):                        # PURA: só depende do parâmetro
    return x * x * x

print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2))           # a expressão lambda da p. 635

def compor(f, g):                   # composição: compor(f, g)(x) = f(g(x))
    return lambda x: f(g(x))

f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4])))      # aplicar-a-todos (p. 636)
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))

notas = [7, 3, 9]
ordenadas = sorted(notas)           # cria uma lista NOVA
print("7:", notas, ordenadas)
notas.sort()                        # altera a lista original
print("8:", notas)
```




```python
contador = 0

def proximo_id(prefixo):            # IMPURA: depende de um estado que muda
    global contador
    contador += 1
    return f"{prefixo}-{contador}"

def cubo(x):                        # PURA: só depende do parâmetro
    return x * x * x

print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2))           # a expressão lambda da p. 635

def compor(f, g):                   # composição: compor(f, g)(x) = f(g(x))
    return lambda x: f(g(x))

f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(g, f)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4])))      # aplicar-a-todos (p. 636)
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))

notas = [7, 3, 9]
ordenadas = sorted(notas)           # cria uma lista NOVA
print("7:", notas, ordenadas)
notas.sort()                        # altera a lista original
print("8:", notas)
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque ela usa a variável global contador, que muda a cada chamada. Mesma entrada ("A") dá resultados diferentes. Isso dificulta os testes porque o resultado depende do estado anterior, não só dos parâmetros.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(12) = 14
compor(g, f)(4) = g(f(4)) = g(6) = 18
A ordem da composição muda o resultado: uma aplica g primeiro, a outra aplica f primeiro.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma lista nova e não altera a original. notas.sort() altera a lista original. O estilo funcional é o sorted, porque não tem efeito colateral (não modifica a lista que já existia).

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

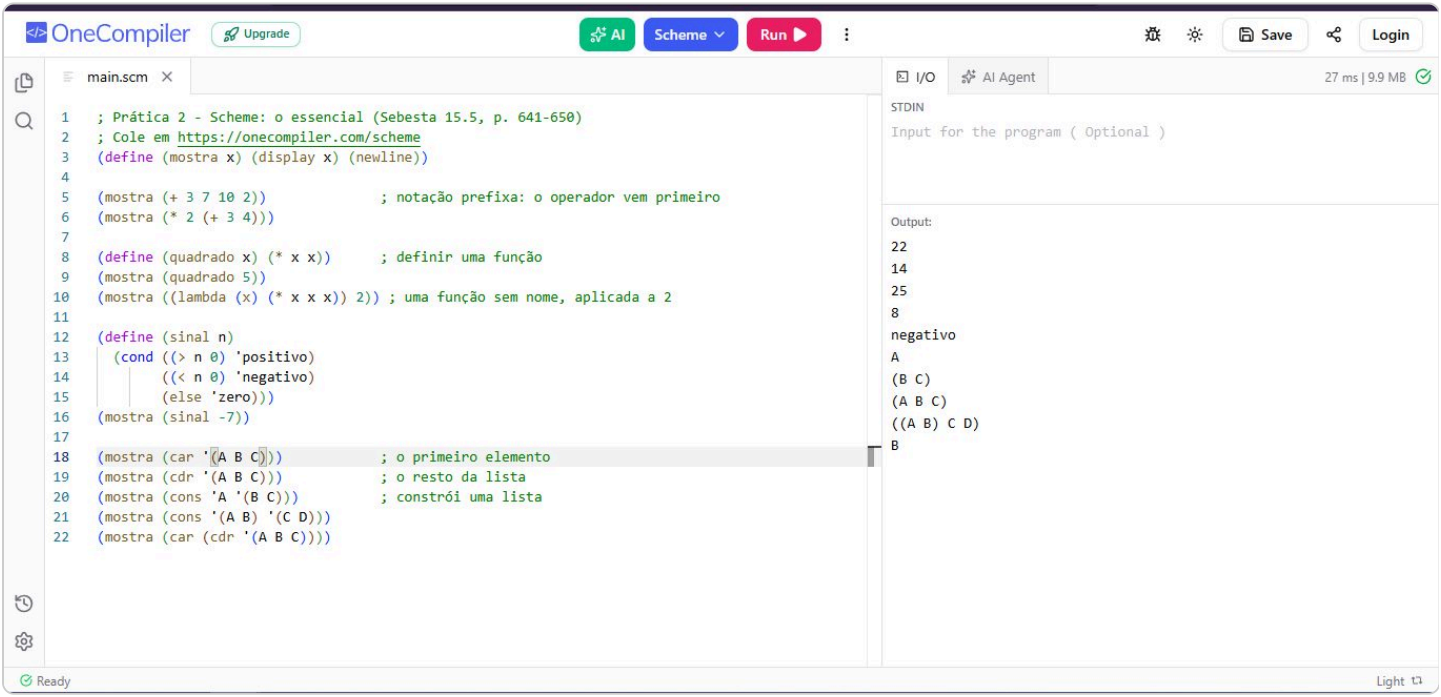
Previsão: (cons '(A B) '(C D))

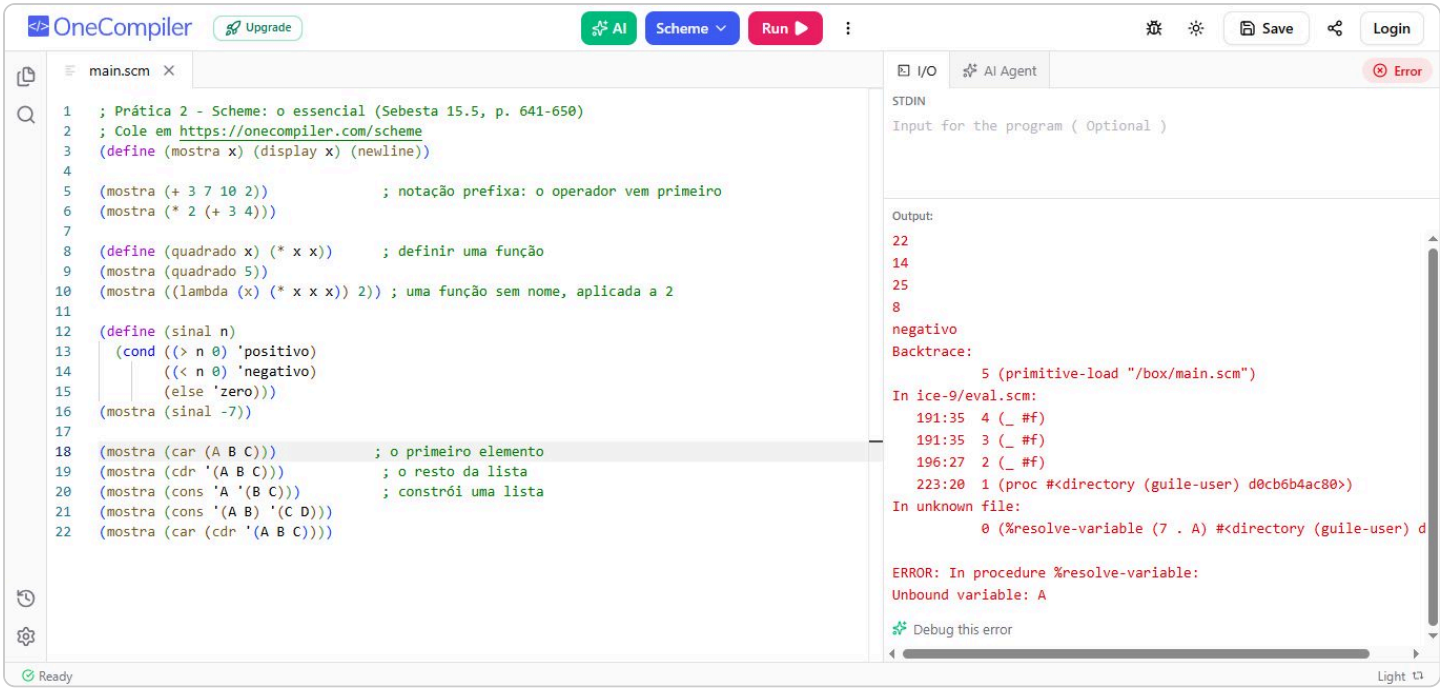
((A B) C D)

Saídas (original e sem o apóstrofo)

```
Original:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

Sem o apóstrofo:
erro (A is not a function / invalid application)
```





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') serve para dizer ao Scheme: "não avalie isso, trate como dado (lista/símbolo)". Sem ele, o Scheme tentou usar A como função e B e C como parâmetros, e deu erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

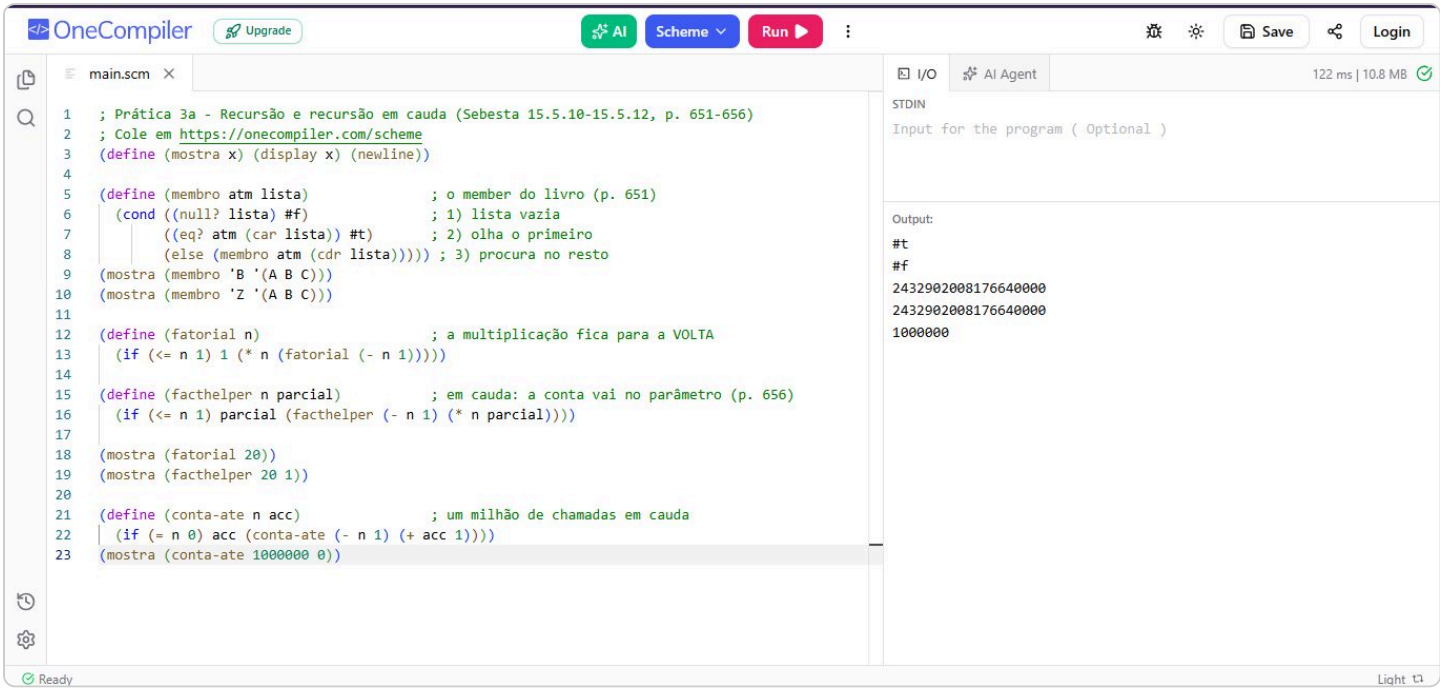
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Scheme deve funcionar (otimiza recursão em cauda). Python deve dar RecursionError no 1 milhão.

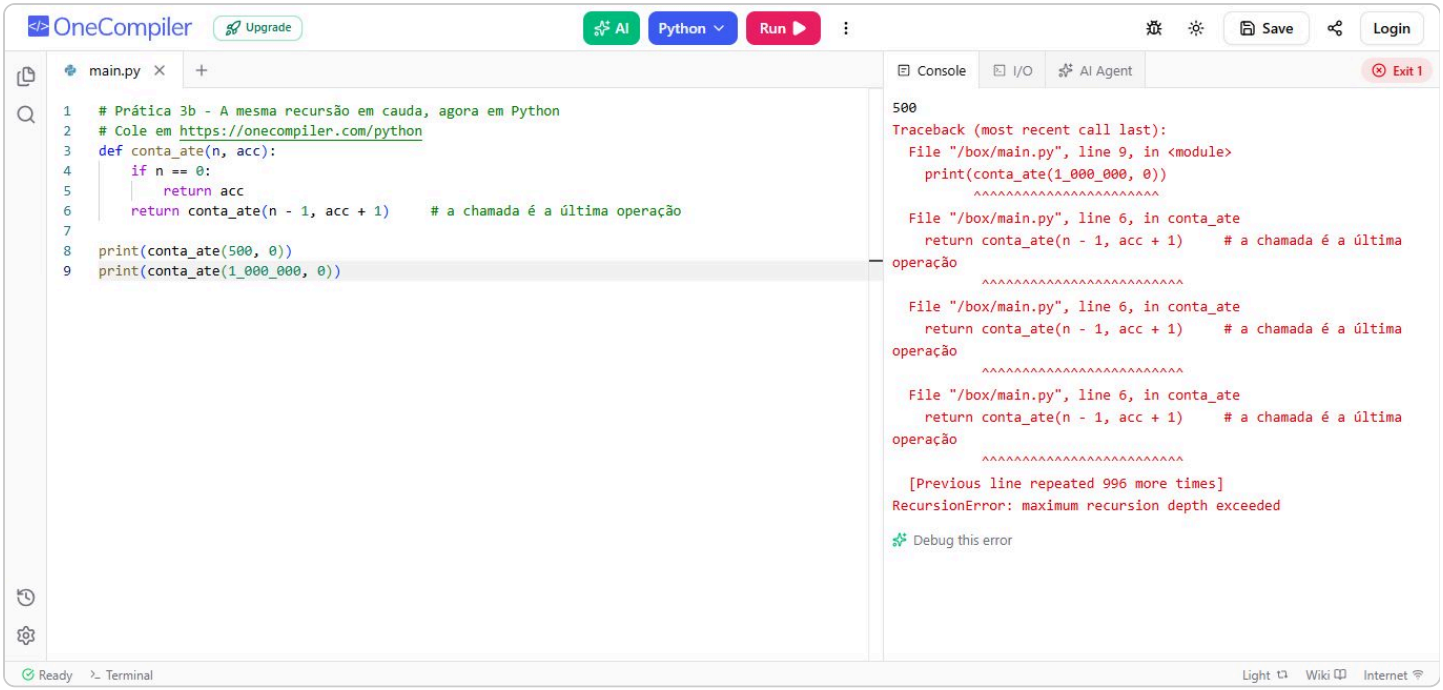
Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```



Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a pilha de chamadas (cada chamada espera a multiplicação da volta). Na recursão em cauda a chamada é a última operação, então a pilha não precisa crescer: o sistema pode reaproveitar o mesmo registro.

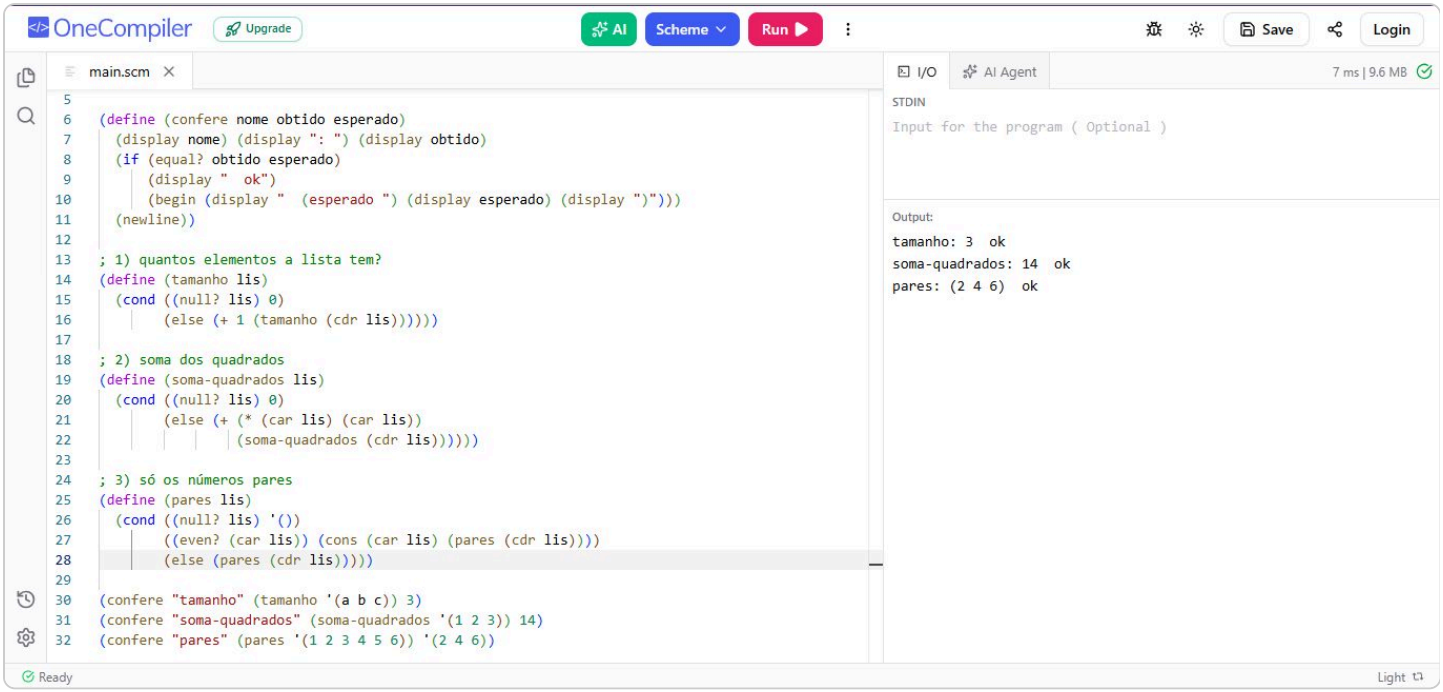
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza a recursão em cauda e reaproveita o registro de ativação, então 1 milhão de chamadas cabem. O Python não faz essa otimização e tem um limite de profundidade de recursão, por isso estoura com RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```




```
(define (confere nome obtido esperado)
  (display nome) (display ": ") (display obtido)
  (if (equal? obtido esperado)
      (display "  ok")
      (begin (display "  (esperado ") (display esperado) (display ")")))
  (newline))

; 1) quantos elementos a lista tem?
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

; 2) soma dos quadrados
(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))

; 3) só os números pares
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))

(confere "tamanho" (tamanho '(a b c)) 3)
(confere "soma-quadrados" (soma-quadrados '(1 2 3)) 14)
(confere "pares" (pares '(1 2 3 4 5 6)) '(2 4 6))
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi lembrar de tratar a lista vazia primeiro (null?). Em todas usei null? no caso base, car para pegar o elemento atual e cdr para chamar de novo no resto da lista. Em pares ainda usei even? e cons para montar a lista de resposta.

Prática 5: Haskell: padrões, compreensões e preguiça

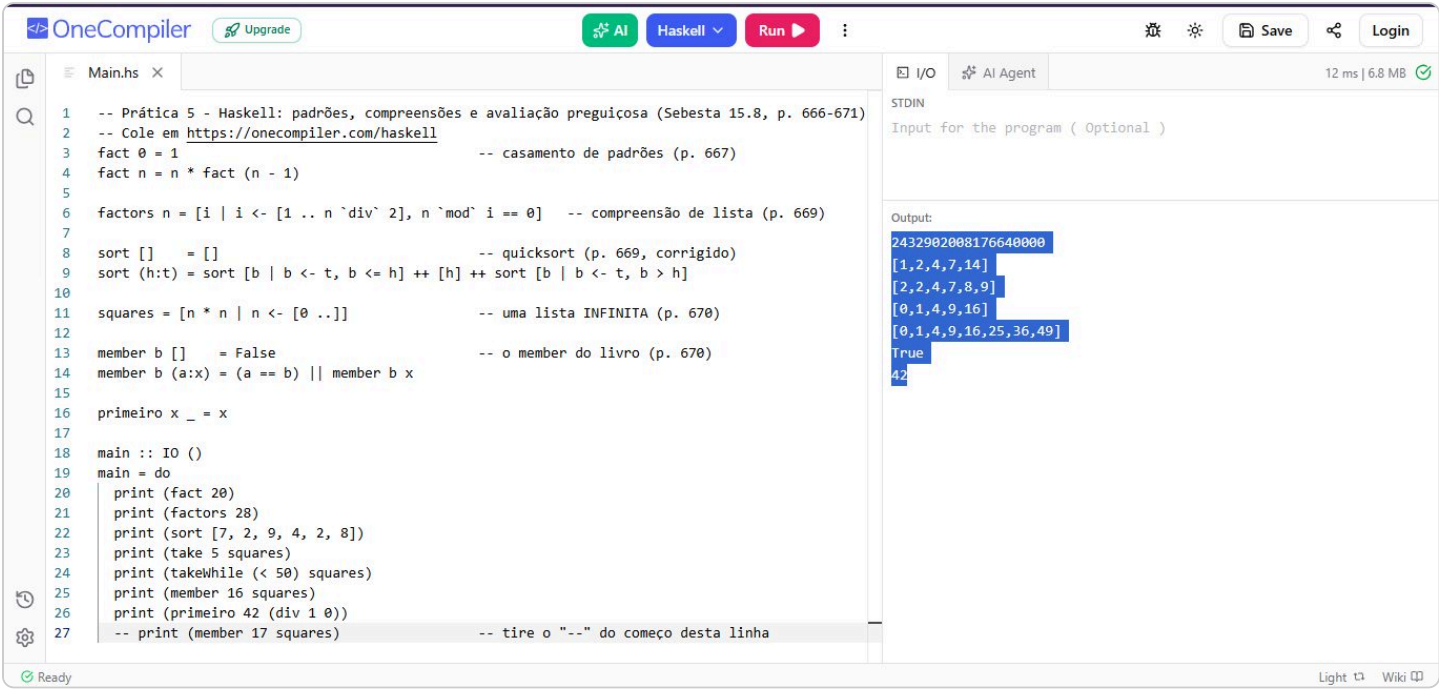
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

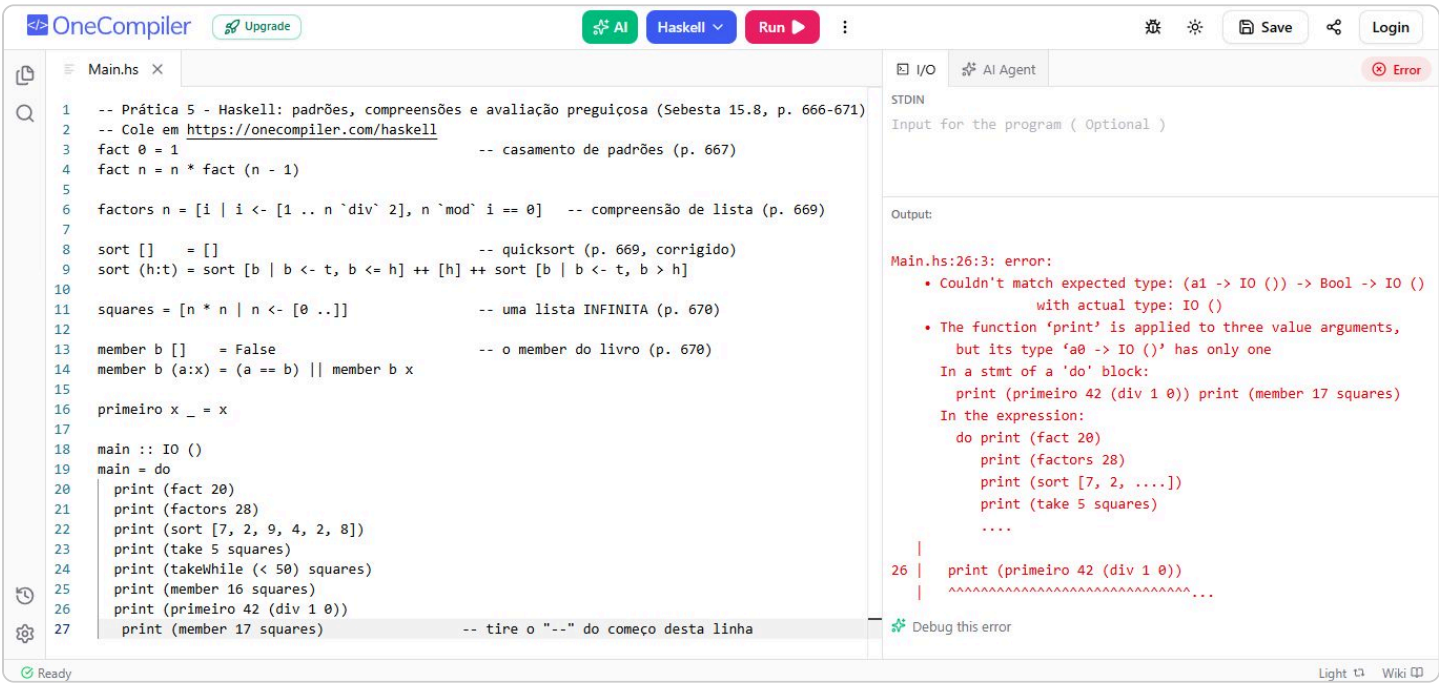
take 5 squares deve imprimir [0,1,4,9,16].
primeiro 42 (div 1 0) deve imprimir 42 (sem erro, porque o Haskell é preguiçoso e não calcula o que não usa).

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
```

```
42
Com member 17:
não termina (lista infinita e 17 nunca aparece como quadrado)
```





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa: só calcula o que precisa. take 5 squares pede só os 5 primeiros elementos, então a lista infinita não é calculada toda. primeiro 42 (div 1 0) não deu erro porque a função ignora o segundo parâmetro (_), então a divisão por zero nunca é executada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 nunca termina porque 17 não é um quadrado perfeito e a lista é infinita: ele continua procurando para sempre. member 16 terminou porque $16 = 4^2$ e aparece na lista. Para corrigir, dá para parar a busca quando o

elemento da lista já for maior que o valor procurado (já que squares está em ordem crescente).

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) deve imprimir 15

[1, 2, 3].map(vezes(5)) deve imprimir [5, 10, 15]

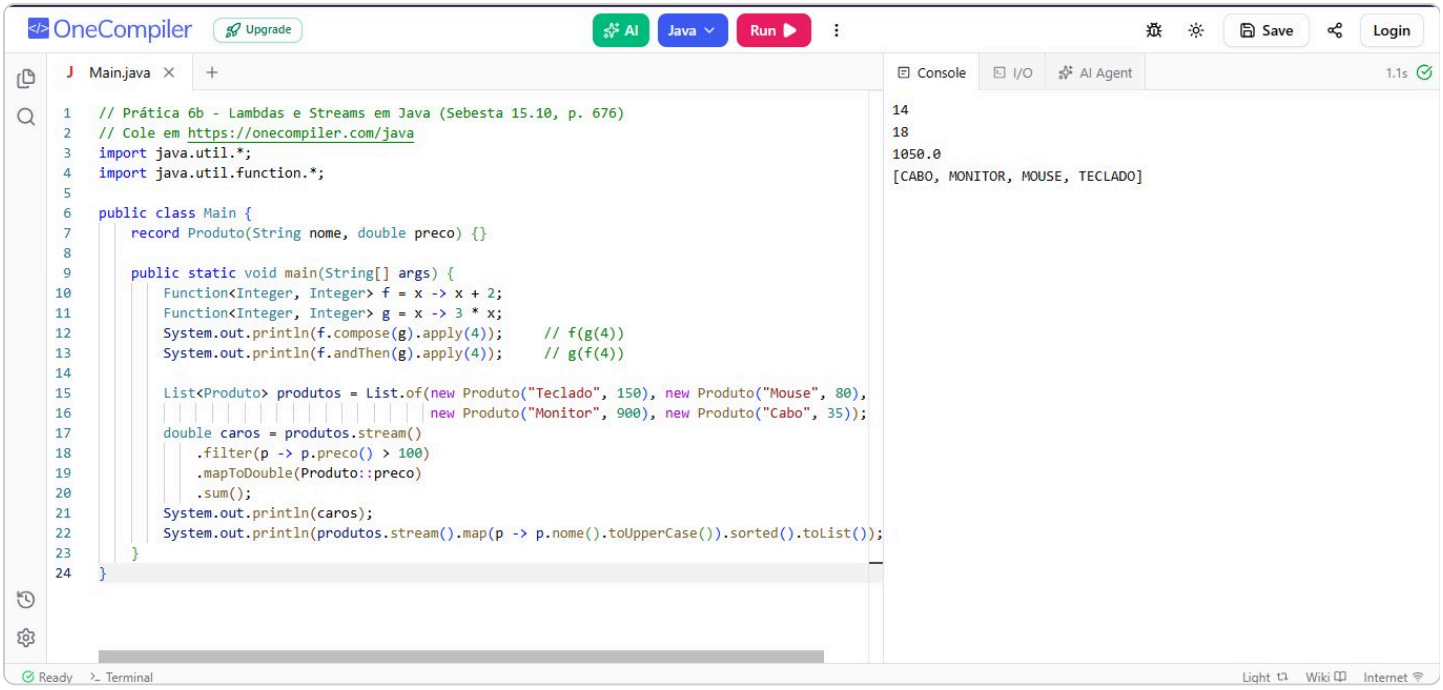
Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```



Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Porque a função devolvida é um fechamento: ela guarda uma ligação com o ambiente onde foi criada e continua "lembrando" o valor de n (o 10), mesmo depois que somador já terminou.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g).apply(4) = f(g(4)) = f(12) = 14
f.andThen(g).apply(4) = g(f(4)) = g(6) = 18
O compose corresponde ao compor(f, g) da Prática 1 (aplica g primeiro, depois f).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
// Prática 7 - Exercício 1: reescreva o laço como um pipeline (stream)
// Cole em https://onecompiler.com/java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = 0;
        for (int n : nums) {
            if (n % 2 == 0) {
                soma += n * n;
            }
        }
        System.out.println(soma);
    }
}
```

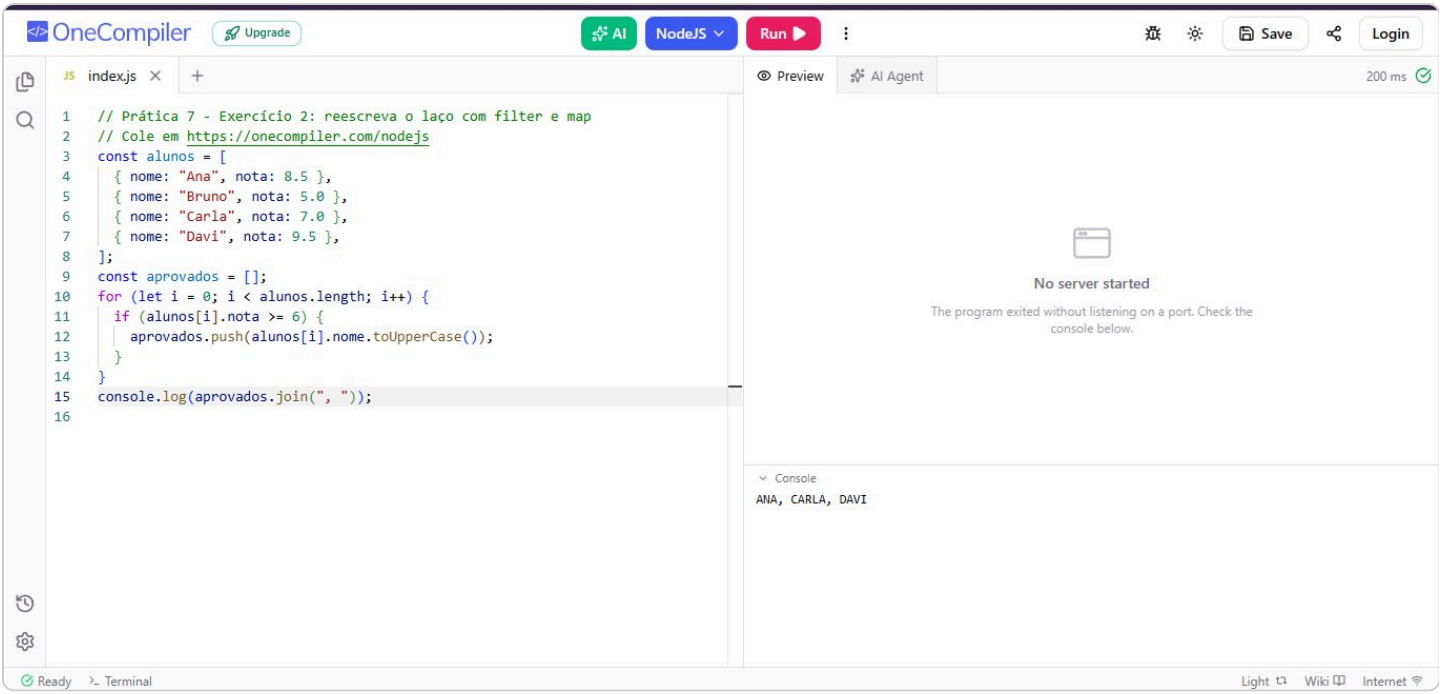
7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
// Prática 7 - Exercício 2: reescreva o laço com filter e map
// Cole em https://onecompiler.com/nodejs
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
const aprovados = [];
for (let i = 0; i < alunos.length; i++) {
  if (alunos[i].nota >= 6) {
    aprovados.push(alunos[i].nome.toUpperCase());
  }
}
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler porque mostra a intenção (filtrar, transformar, somar) sem controlar o laço. A versão com laço é mais fácil de depurar, porque dá para colocar um print no meio e ver o valor de cada passo.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque descreve o que se quer (lista de 1 a n, elevar ao cubo, somar) sem controlar o laço. Eu aceitaria o custo extra de tempo em sistemas onde clareza e manutenção importam mais que performance máxima (ex.: ferramentas internas, protótipos). Não aceitaria em sistemas de tempo real ou com carga muito alta.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o Haskell conseguir trabalhar com listas infinitas e não calcular o que não precisa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão é b <- h em vez de b <= h (ou b < h). O compilador reclama de tipo porque trata h como se fosse uma lista (por causa do <-), quando deveria ser uma comparação. Correção: b <= h.